

Practica 4

Concurrencia

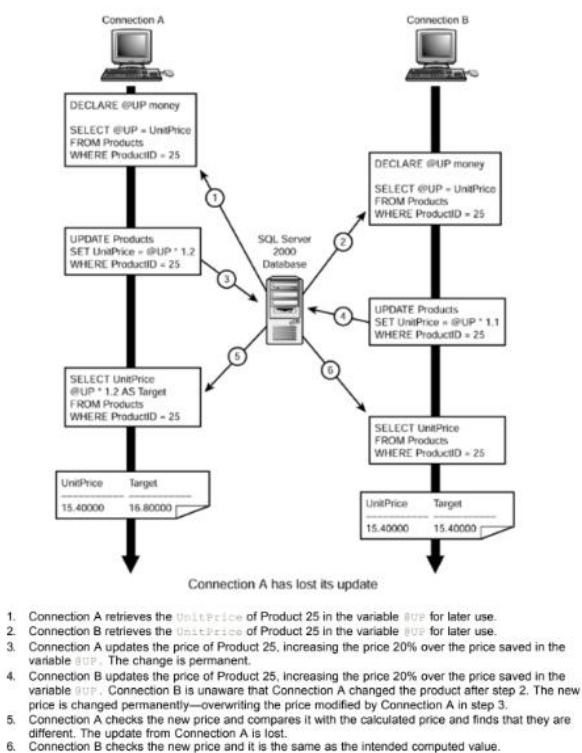
“Base de Datos Distribuidas”

- Peralta Ordoñez Jesús
- Valencia Cedeño Marcos Gael
- Vargas Clemente Leonel Zajid

Actualización Perdida (Lost Update)

Lost Updates

You can experience a lost update situation whenever two connections modify the same data in sequence, because SQL Server will maintain only the last successful update. Consider the example of [Figure 13.1](#):



This problem can be prevented by writing atomic `UPDATE` statements in both Connection A and Connection B. An atomic `UPDATE` contains the reading operation and the writing operation in a single statement, as in [Listing 13.10](#).

Descripción del problema

Una situación de "Actualización Perdida" ocurre cuando dos transacciones concurrentes leen el mismo dato, realizan un cálculo basado en esa lectura y luego intentan guardar los cambios de forma secuencial. El problema radica en que la segunda transacción guarda su resultado sin ser consciente de que la primera transacción ya había modificado el registro original. Como SQL Server mantiene únicamente la última actualización exitosa, los cambios realizados por la primera conexión se sobrescriben y se pierden de forma permanente, sin que el sistema arroje ningún mensaje de error.

Código de ejemplo (Simulación del error)

Para demostrar este problema en la base de datos Northwind, se simulamos dos conexiones utilizando variables locales y retardos (WAITFOR DELAY) para forzar la concurrencia.

```
USE Northwind;
GO

PRINT '--- ESTADO ANTES DEL REINICIO ---';
SELECT ProductID, ProductName, UnitPrice
FROM Products
WHERE ProductID = 25;

-- Reiniciamos el precio del producto 25 a su valor base (15.40)
UPDATE Products
SET UnitPrice = 15.40
WHERE ProductID = 25;
```

	ProductID	ProductName	UnitPrice
1	25	NuNuCa Nuß-Nougat-Creme	15.40

Proceso A (Conexión 1)

```
1 USE Northwind;
2 GO
3 DECLARE @UP money;
4 -- 1. Recupera el precio actual (15.40)
5 SELECT @UP = UnitPrice FROM Products WHERE ProductID = 25;
6
7 -- Simula un tiempo de procesamiento, permitiendo que el Proceso B también lea el 15.40
8 WAITFOR DELAY '00:00:10';
9
10 -- 3. Actualiza el precio incrementando un 20%
11 UPDATE Products
12 SET UnitPrice = @UP * 1.2
13 WHERE ProductID = 25;
14 -- El precio aquí sube a 18.48
15 GO
```

25 % ● No se encontraron problemas. Línea: 1, Ca

Mensajes

(1 fila afectada)

Hora de finalización: 2026-09-22T18:32:42.0130212-06:00

Proceso B (Conexión 2 - Ejecutado inmediatamente después del Proceso A)

```

1  USE Northwind;
2  GO
3  WAITFOR DELAY '00:00:02'; -- Asegura que A lea primero
4  DECLARE @UP money;
5
6  -- 2. Recupera el mismo precio desactualizado (15.40)
7  SELECT @UP = UnitPrice FROM Products WHERE ProductID = 25;
8
9  -- Espera a que el Proceso A termine de guardar su cambio
10 WAITFOR DELAY '00:00:12';
11
12 -- 4. Actualiza el precio incrementando un 10% sobre el valor que leyó
13 UPDATE Products
14 SET UnitPrice = @UP * 1.1
15 WHERE ProductID = 25;
16 -- El precio final queda en 16.94, sobrescribiendo y perdiendo el 18.48 del Proceso A
17 GO

```

105 % ✓ No se encontraron problemas. Línea:

Mensajes

(1 fila afectada)

Hora de finalización: 2026-09-22T18:32:49.9817161-06:00

Visualización de lo que ocurrió

```

16 PRINT '--- ESTADO LISTO PARA LA PRUEBA ---';
17 SELECT ProductID, ProductName, UnitPrice
18 FROM Products
19 WHERE ProductID = 25;
20 GO

```

05 % ✓ No se encontraron problemas.

Resultados Mensajes

ProductID	ProductName	UnitPrice
25	NuNuCa Nuß-Nougat-Creme	16.94

Código de Solución (Actualización Atómica):

Para prevenir este problema, la operación debe realizarse de forma atómica integrando la lectura y escritura en una sola instrucción.

```
USE Northwind;  
GO  
UPDATE Products  
SET UnitPrice = UnitPrice * 1.2  
WHERE ProductID = 25;  
GO
```

Conclusiones del análisis

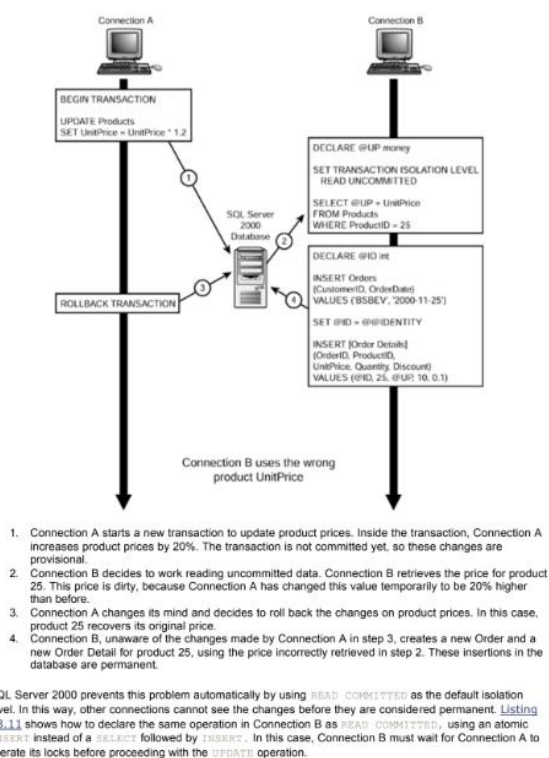
Al analizar este escenario, se concluye que el problema de la actualización perdida es un fallo de lógica de programación y no un error del motor de la base de datos. Se origina al separar las operaciones de lectura (asignando el valor a una variable @UP) y escritura (haciendo el UPDATE con dicha variable) en pasos distintos.

Al implementar una actualización atómica ($\text{UnitPrice} = \text{UnitPrice} * 1.2$), el gestor de la base de datos evalúa y bloquea el registro en el instante exacto de la modificación. Esto asegura que cualquier otra transacción concurrente deba esperar a que se libere el bloqueo, garantizando que los cálculos siempre se realicen sobre el dato más reciente y manteniendo la integridad de la base de datos.

Uncommitted Dependency (Dirty Read)

Reading data without using locks can produce unexpected results. Other connections can modify data temporarily, and you can use that new data in your calculations, driving you to incorrect results. In this case, you could say that you are reading dirty data, because the data has not been committed yet. In other words, your calculations depend on uncommitted data.

Figure 13.2 shows an example of this situation:



Descripción del problema

Este problema ocurre porque leer datos sin usar bloqueos puede producir resultados inesperados. Otras conexiones pueden modificar datos de forma temporal, y si otra conexión utiliza esos nuevos datos no confirmados en sus cálculos, se llegará a resultados incorrectos. A esto se le conoce como leer "datos sucios" (dirty data), ya que los cálculos terminan dependiendo de información que aún no ha sido confirmada de manera definitiva en la base de datos.

Código de ejemplo (Simulación del error)

Para este ejemplo, nuevamente abriremos dos ventanas (conexiones). La Conexión B forzará intencionalmente una lectura sucia cambiando su nivel de aislamiento a `READ UNCOMMITTED`.

```
USE Northwind;
GO
-- Reiniciamos el precio del producto 25 a su valor base (15.40)
UPDATE Products SET UnitPrice = 15.40 WHERE ProductID = 25;
PRINT 'El precio inicial del producto 25 es 15.40';
GO
```

Visualización antes de ejecutar los procesos

```
PRINT '--- 1. ESTADO FINAL DEL PRODUCTO 25 ---';
SELECT ProductID, ProductName, UnitPrice
FROM Products
WHERE ProductID = 25;
```

Resultados		Mensajes	
	ProductID	ProductName	UnitPrice
1	25	NuNuCa Nuß-Nougat-Creme	15.40

Proceso A (Conexión 1)

```
USE Northwind;
GO
-- 1. La Conexión A inicia una nueva transacción para actualizar los precios.
/ BEGIN TRANSACTION;

-- Dentro de la transacción, la Conexión A incrementa los precios en un 20%.
UPDATE Products SET UnitPrice = UnitPrice * 1.2 WHERE ProductID = 25;

PRINT 'Proceso A: Precio actualizado temporalmente a 18.48. Esperando 10 segundos...';
-- La transacción aún no está confirmada, por lo que estos cambios son provisionales.
WAITFOR DELAY '00:00:10';

-- 3. La Conexión A cambia de opinión y decide deshacer los cambios con un rollback.
ROLLBACK TRANSACTION;

PRINT 'Proceso A: ROLLBACK ejecutado. El producto 25 recupera su precio original (15.40).';
-- En este caso, el producto 25 recupera su precio original.
GO
```

Proceso B (Conexión 2 - Ejecutar inmediatamente después de iniciar el Proceso A)

```

-- Proceso B
USE Northwind;
GO
DECLARE @UP money;

-- 2. La Conexión B decide trabajar leyendo datos no confirmados.
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;

WAITFOR DELAY '00:00:02'; -- Damos 2 segundos para asegurar que A ya hizo su UPDATE provisional

-- La Conexión B recupera el precio del producto 25.
-- Este precio es "sucio" porque la Conexión A lo cambió temporalmente para ser 20% más alto.
SELECT @UP = UnitPrice FROM Products WHERE ProductID = 25;
PRINT 'Proceso B: Precio sucio leído = ' + CAST(@UP AS VARCHAR);

WAITFOR DELAY '00:00:10'; -- Esperamos a que A haga el ROLLBACK para demostrar el error

DECLARE @ID int;
-- 4. La Conexión B, sin saber de los cambios de la Conexión A, crea una nueva Orden.
-- (Usamos el cliente 'VINET' que viene por defecto en Northwind)
INSERT Orders (CustomerID, OrderDate) VALUES ('VINET', '2000-11-25');
SET @ID = @@IDENTITY;

-- Crea un nuevo Detalle de Orden para el producto 25 usando el precio recuperado incorrectamente.
INSERT [Order Details] (OrderID, ProductID, UnitPrice, Quantity, Discount)
VALUES (@ID, 25, @UP, 10, 0.1);

PRINT 'Proceso B: Inserción permanente realizada con el precio incorrecto (18.48).';
-- Estas inserciones en la base de datos son permanentes.
GO

```

Visualización de lo que sucedió

```

PRINT '--- 2. EVIDENCIA DE LA LECTURA SUCIA (NUEVA ORDEN) ---';
-- Buscamos el último detalle de orden que se insertó para el producto 25
SELECT TOP 1
    od.OrderID,
    o.OrderDate,
    od.ProductID,
    p.ProductName,
    od.UnitPrice AS PrecioCobradoEnLaOrden
FROM [Order Details] od
JOIN Orders o ON od.OrderID = o.OrderID
JOIN Products p ON od.ProductID = p.ProductID
WHERE od.ProductID = 25
ORDER BY od.OrderID DESC;
/*
Aquí está el error: El "PrecioCobradoEnLaOrden" será 18.48.
El Proceso B leyó el dato fantasma antes del ROLLBACK y guardó
una venta de forma permanente con un precio que realmente
ya no existe en la base de datos.
*/
GO

```

▼ ✔ No se encontraron problemas.

Resultados Mensajes

OrderID	OrderDate	ProductID	ProductName	PrecioCobradoEnLaOrden
1078	2000-11-25 00:00:00.000	25	NuNuCa Nuß-Nougat-Creme	18.48

Código de Solución (Nivel READ COMMITTED y operación atómica)

```
USE Northwind;
GO
-- SQL Server previene este problema automáticamente usando READ COMMITTED como nivel de aislamiento por defecto.
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

DECLARE @ID int;

INSERT Orders (CustomerID, OrderDate) VALUES ('VINET', '2000-11-25');
SET @ID = @@IDENTITY;

-- Se utiliza un INSERT atómico en lugar de un SELECT seguido de un INSERT.
INSERT [Order Details] (OrderID, ProductID, UnitPrice, Quantity, Discount)
SELECT @ID, ProductID, UnitPrice, 10, 0.1
FROM Products
WHERE ProductID = 25;
GO
```

Conclusiones del análisis

Al permitir explícitamente el nivel de aislamiento READ UNCOMMITTED, vulneramos la integridad de los datos, ya que permitimos que el sistema lea un registro que estaba "en el aire" dentro de una transacción abierta. Cuando esa transacción decidió hacer un ROLLBACK, nuestro Proceso B se quedó trabajando con un valor fantasma que ya no existe en la base de datos y lo guardó de forma permanente en otra tabla.

Al utilizar el nivel de aislamiento predeterminado READ COMMITTED, SQL Server asegura que otras conexiones no puedan ver los cambios antes de que sean considerados permanentes. Bajo esta configuración correcta, el Proceso B debe esperar forzosamente a que el Proceso A libere sus bloqueos (ya sea mediante un COMMIT o un ROLLBACK) antes de poder proceder con la lectura y la operación de actualización o inserción.

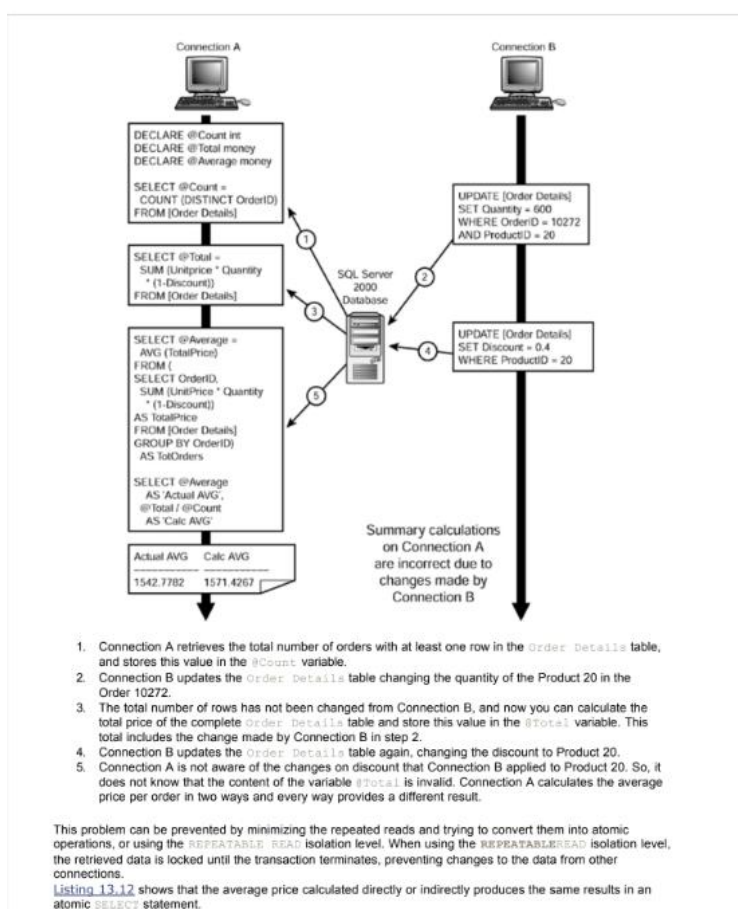
Análisis Inconsistente (Lectura No Repetible / Nonrepeatable Read)

Inconsistent Analysis (Nonrepeatable Read)

In a multiuser environment, other users dynamically modify the data stored in a database. Trying to execute a long-running process, such as a monthly report, can produce some inconsistencies because of changes produced in the database from the beginning of the report to the end. This can be considered an inconsistent analysis, because every time you read data, the data is different. This situation is called *nonrepeatable reads*.

To produce a long-running report, you must retrieve the number of orders from the `Order Details` table because you are interested only in orders with Order Details. Later in the report, you might want to calculate the average of total price per order, using the value retrieved previously.

Figure 13.3 shows an example of this situation:



Descripción del problema

En un entorno multiusuario, otros usuarios modifican dinámicamente los datos almacenados en una base de datos. Intentar ejecutar un proceso de larga duración, como un reporte mensual, puede producir inconsistencias debido a los cambios que ocurren en la base de datos desde el principio hasta el final del reporte. Esto se considera un análisis inconsistente porque cada vez que se leen

los datos, la información es diferente. A esta situación se le llama lecturas no repetibles.

Código de ejemplo (Simulación del error)

```
USE Northwind;
GO
-- Restauramos los valores del producto 20 para que la prueba sea exacta
UPDATE [Order Details]
SET Quantity = 20, Discount = 0
WHERE OrderID = 10272 AND ProductID = 20;

UPDATE [Order Details]
SET Discount = 0
WHERE ProductID = 20;
GO
```

Visualización antes de los procesos

```
PRINT '----- ESTADO INICIAL DEL PRODUCTO 20 (ANTES DE LOS PROCESOS) -----';
SELECT OrderID, ProductID, UnitPrice, Quantity, Discount
FROM [Order Details]
WHERE ProductID = 20;
/*
Al ejecutar esto, debes notar dos cosas importantes:
1. En la Orden 10272, la columna "Quantity" (Cantidad) es exactamente 20.
2. En todos los registros, la columna "Discount" (Descuento) es 0.00.
Este es el estado puro con el que el Proceso A espera hacer sus cálculos.
*/
GO
```

%  No se encontraron problemas.

Resultados  Mensajes

OrderID	ProductID	UnitPrice	Quantity	Discount
10252	20	64.80	40	0
10272	20	64.80	20	0
10292	20	64.80	20	0
10372	20	64.80	12	0
10384	20	64.80	28	0
10514	20	81.00	39	0
10523	20	81.00	15	0
10593	20	81.00	21	0
10660	20	81.00	21	0
10828	20	81.00	5	0
10878	20	81.00	20	0
10927	20	81.00	5	0
10953	20	81.00	50	0
10986	20	81.00	15	0
11021	20	81.00	15	0
11077	20	81.00	1	0

Proceso A (Conexión 1 - El Reporte)

```
USE Northwind;
GO
DECLARE @Count int;
DECLARE @Total money;
DECLARE @Average money;

-- 1. La Conexión A recupera el número total de órdenes y lo guarda en la variable @Count.
SELECT @Count = COUNT(DISTINCT OrderID) FROM [Order Details];

-- Pausamos para permitir que el Proceso B modifique las cantidades
WAITFOR DELAY '00:00:05';

-- 3. Se calcula el precio total y se almacena en la variable @Total.
-- Este total ya incluye el cambio realizado por la Conexión B en su paso 2.
SELECT @Total = SUM(UnitPrice * Quantity * (1 - Discount)) FROM [Order Details];

-- Pausamos para permitir que el Proceso B modifique los descuentos
WAITFOR DELAY '00:00:05';

-- 5. La Conexión A, sin ser consciente de los cambios en el descuento, calcula el precio promedio por orden de dos maneras.
SELECT @Average = AVG(TotalPrice) FROM (
    SELECT OrderID, SUM(UnitPrice * Quantity * (1 - Discount)) AS TotalPrice
    FROM [Order Details]
    GROUP BY OrderID
) AS TotOrders;

-- Imprimimos la comparación
SELECT
    @Average AS 'Actual AVG',
    @Total / @Count AS 'Calc AVG';
-- Cada manera de calcular el promedio proporcionará un resultado diferente.
GO
```

Proceso B (Conexión 2 - Ejecutar justo después del Proceso A)

```
USE Northwind;
GO
WAITFOR DELAY '00:00:02'; -- Asegura que A cuente las órdenes primero

-- 2. La Conexión B actualiza la tabla cambiando la cantidad del Producto 20 en la Orden 10272.
UPDATE [Order Details]
SET Quantity = 600
WHERE OrderID = 10272 AND ProductID = 20;

WAITFOR DELAY '00:00:05'; -- Espera a que A calcule el Total

-- 4. La Conexión B actualiza la tabla de nuevo, cambiando el descuento del Producto 20.
UPDATE [Order Details]
SET Discount = 0.4
WHERE ProductID = 20;
GO
```

Código de Solución (Operación Atómica)

Este problema se puede prevenir minimizando las lecturas repetidas e intentando convertirlas en operaciones atómicas. El siguiente script demuestra cómo el precio promedio calculado directa o indirectamente produce los mismos resultados utilizando una sola declaración SELECT atómica.

```
USE Northwind;
GO
-- Operación atómica que calcula ambos promedios simultáneamente en el mismo instante de tiempo
SELECT
    AVG(TotalPrice) AS 'Actual AVG',
    SUM(TotalPrice) / COUNT(OrderID) AS 'Calc AVG'
FROM (
    SELECT OrderID, SUM(UnitPrice * Quantity * (1 - Discount)) AS TotalPrice
    FROM [Order Details]
    GROUP BY OrderID
) AS TotOrders;
GO
```

Conclusiones del análisis

El análisis inconsistente ocurre al separar la lógica de un reporte en múltiples consultas independientes a lo largo del tiempo. Los cálculos resumidos en la Conexión A resultan ser incorrectos debido a las modificaciones realizadas simultáneamente por la Conexión B.

Para solucionar esto, además de empaquetar las consultas en una sola instrucción atómica, también se puede utilizar el nivel de aislamiento REPEATABLE READ. Al utilizar el nivel de aislamiento REPEATABLE READ, los datos recuperados quedan bloqueados hasta que la transacción termina, lo que previene que otras conexiones realicen cambios en esos datos.

Lecturas Fantasma (Phantom Reads)

Descripción del problema

Si otras conexiones insertan datos en el rango de los registros que estás analizando, puedes encontrarte con que esas nuevas filas aparecen en tus conjuntos de resultados sin motivo aparente desde el punto de vista de tu conexión. A estas nuevas filas se les llama *fantasmas* (phantoms). Este problema también produce análisis inconsistentes, ya que los totales previos dejan de ser válidos tras la inserción de las nuevas filas

Script de Preparación

```
USE Northwind;
GO
-- Limpiamos el registro de pruebas pasadas para asegurar el entorno
DELETE FROM [Order Details] WHERE OrderID = 10615 AND ProductID = 37;
```

Visualización antes de los procesos

```

-- Ejecución de la consulta
PRINT '--- ESTADO INICIAL (ANTES DE LOS PROCESOS) ---';
-- La Conexión A recupera la lista de órdenes, lo que produce un resultado de 6 filas.
SELECT OrderID, ProductID, UnitPrice
FROM [Order Details]
WHERE ProductID = 37;
GO

```

✓ No se encontraron problemas.

Resultados Mensajes

OrderID	ProductID	UnitPrice
10259	37	20.80
10337	37	20.80
10408	37	20.80
10523	37	26.00
10847	37	26.00
10966	37	26.00

Proceso A (Conexión 1)

```

USE Northwind;
GO
-- 1. Ejecuta la lectura inicial
SELECT OrderID, ProductID, UnitPrice
FROM [Order Details]
WHERE ProductID = 37;

-- Pausamos 10 segundos para dar tiempo a que el Proceso B inserte la nueva fila
WAITFOR DELAY '00:00:10';

-- 3. La Conexión A reenvía la misma instrucción del paso 1, pero recupera una fila nueva.
SELECT OrderID, ProductID, UnitPrice
FROM [Order Details]
WHERE ProductID = 37;
-- Como la Conexión A no estaba al tanto de esta inserción, considera a esta fila como un fantasma.
GO

```

Proceso B (Conexión 2 - Ejecutar inmediatamente después de iniciar el Proceso A)

```
USE Northwind;
GO
-- Damos 3 segundos para asegurar que el Proceso A hizo su primera lectura de 6 filas
WAITFOR DELAY '00:00:03';

-- 2. La Conexión B inserta un nuevo detalle en la Orden 10615 con el producto 37.
INSERT [Order Details] (OrderID, ProductID, UnitPrice, Quantity, Discount)
VALUES (10615, 37, 21.540, 20, 0.1);
PRINT 'Proceso B: Registro insertado exitosamente.';
GO
```

Script de Visualización (Después)

Realmente, la visualización del error ocurre dentro de la misma ventana del Proceso A, ya que la segunda tabla que imprime mostrará 7 filas en lugar de 6, incluyendo la orden 10615

	OrderID	ProductID	UnitPrice
1	10259	37	20.80
2	10337	37	20.80
3	10408	37	20.80
4	10523	37	26.00
5	10847	37	26.00
6	10966	37	26.00

	OrderID	ProductID	UnitPrice
1	10259	37	20.80
2	10337	37	20.80
3	10408	37	20.80
4	10523	37	26.00
5	10615	37	21.54
6	10847	37	26.00
7	10966	37	26.00

Código de Solución

Prevenir este problema es más difícil que en los casos anteriores, porque SQL Server necesita bloquear filas que aún no existen (bloquear datos inexistentes). Para solucionarlo, se puede utilizar el nivel de aislamiento SERIALIZABLE o la sugerencia de optimizador (optimizer hint) SERIALIZABLE.

```

51
52 USE Northwind;
53 GO
54 -- Al utilizar la sugerencia SERIALIZABLE, SQL Server bloquea el rango completo de datos.
55 -- Si el Proceso B intenta hacer un INSERT mientras esto ocurre, se quedará en espera.
56 SELECT OrderID, ProductID, UnitPrice
57 FROM [Order Details] (SERIALIZABLE)
58 WHERE ProductID = 37;
59 GO

```

%  No se encontraron problemas.

Resultados  Mensajes

OrderID	ProductID	UnitPrice
10259	37	20.80
10337	37	20.80
10408	37	20.80
10523	37	26.00
10615	37	21.54
10847	37	26.00
10966	37	26.00

Conclusiones del análisis

Tener lecturas fantasmas no es algo que deba evitarse por sí solo, a menos que produzcan análisis inconsistentes, lo cual suele ser el caso habitual. Al presentarse inserciones de otras conexiones en rangos de datos que estamos leyendo, nuestras segundas lecturas arrojarán cantidades distintas. Implementar el nivel SERIALIZABLE soluciona el problema al colocar bloqueos de rango de claves, garantizando que ninguna otra transacción pueda insertar filas en el rango que nuestra consulta abarca hasta que terminemos de procesar